ENGINEERING NOTES

How the Timesheeter app is put together

A tour of the main sections and the machinery behind them — how jobs, timesheets and Wrike stay on one page. Pairs with the job-data technical notes, which cover the resolution pipeline in full.

1 The shape of the app

The app is a React single-page application (Vite, Tailwind) talking to two back-ends. Supabase (Postgres behind PostgREST) is the database; Wrike is where the work actually lives. Each main section is its own route and its own bundle, so a user sitting in the timesheet doesn't pull down the canvas or the admin screens.

Wrike access goes through a small worker that owns the OAuth token and proxies the Wrike API. The browser never holds a Wrike credential — it asks the worker, and the worker talks to Wrike. That keeps the token in one place and out of the page.

Section	What it does	Main data
Tracker	The day-to-day log — a Mon–Sun week grid, a timer, manual hours, and pulling your Wrike timelogs.	`tasks` + Wrike timelogs
The timesheet we use	The studio's working timesheet. Pulls timelogs, groups them by task and day, lets you edit and export.	`tasks` + Wrike timelogs
Management	Job Book, film setup, and the Wrike scan that keeps the book in step with Wrike's folder tree.	`jobs`, `films`, Wrike folders
Canvas	Motion and launch tracking boards, fed by the Wrike mirror.	`wrike_tasks_cache`
Profile / Admin	Your profile, position and rate; team administration for the management allowlist.	`profiles`, `positions`

2 One job, one code

Everything keys on the **XY code**, not the human-readable string. A job number arrives in several shapes and is reduced to the same thing before it is ever used as a key:

```
// src/utils/wrikeHelpers.js — jobKey()
"XY025716"                          // bare code
"XY025716_LUG_D6"                   // suffix → still XY025716
"The Odyssey : XY025716, Finishing"   // canonical → still XY025716

const jobKey = (jobNumber) => {
  const m = String(jobNumber).match(/XY\d{5,6}/i);
  return m ? m[0].toUpperCase() : String(jobNumber).trim();   // fallback: free-text internal jobs
};
```

The database enforces the same rule — a unique index on the code, wherever it appears in the job number:

```
-- supabase/schema.sql — jobs_job_code_key
create unique index jobs_job_code_key
  on public.jobs (substring(job_number from 'XY\d{5,6}'));
```

Two stores carry the day-to-day. The **Job Book** (`jobs`) is the curated, admin-edited record — it holds the canonical `"Film : XY…, Description"` string plus a resolved `film_title` and `client`. The **timesheet rows** (`tasks`) are one row per entry, each carrying `job_number`, `territory`, `film_title` and `client` as read at the moment it was logged, plus `source` (`"tracker"` or `"legacy"`). Because both sides key on the code, a bare old row, a suffixed folder, and the curated canonical string all land on the same job, and the same job can't be registered twice under two spellings.

3 The Tracker

The Tracker shows the current week as a Mon–Sun grid. Each cell is a logging surface: pick a job, choose a film and a country, enter time — a timer, or hours and minutes by hand. Logging writes the row with the job, film, and country resolved through the pipeline in §6, and stamps it with the date of the *selected* weekday, so backfilling Monday on a Wednesday records Monday's date, not Wednesday's.

A pull brings in your Wrike timelogs for the day. Each timelog has an id; the app keeps a set of the ids it has already stored and skips anything in it, so a pull never writes the same entry twice. The set is built by reading *every* row the user has ever pulled that carries a timelog id — deliberately, because the same timelog id can otherwise slip back in and create a duplicate.

All rows on both timesheet surfaces go through one store (`src/hooks/useTasks.js`). The database keeps time as an `H:MM` string; in memory the app keeps the exact seconds. The conversion happens at the boundary, so what you see in the day total and what you export are the same number.

---

4 The timesheet we use

This is the working timesheet. It pulls your Wrike timelogs for today and yesterday, groups them by task and by day, sums the hours for each group, and writes one row per group. A row that aggregates several same-day logs keeps all their ids (`"id1,id2,id3"`), so a later pull of any of them is recognised and skipped.

Each pulled task runs the same resolution as the Tracker — job number, film, country, client — and the resulting row carries the resolved values. The grid is editable: time, job, film, country and notes can be corrected by hand, and the corrections stay put because the row is stored with what you set, not re-guessed on every render. The export produces the company timesheet format, with the country list rendered the way the company template expects it.

---

5 Management — the Job Book and the Wrike scan

Management is where jobs are curated. The Job Book is the admin-edited registry: searchable, filterable by month, with per-row editing and bulk operations. Film setup groups films by studio so the pickers stay navigable as the list grows.

The centrepiece is the Wrike scan. It walks Wrike's folder tree once, finds every folder whose title carries an XY code, and reads the description, film and studio arm off the folder's name and its place in the tree. Real job folders sit at `Studio space → <Studio> → <Film> → <Job>`, so the film is the deepest non-year folder below the studio, and the studio's title decides the client and the region prefix.

FOR EVERY FOLDER WHOSE TITLE CARRIES AN XY CODE

1. Take the **code**.
2. Read the **description** from the folder-title suffix after the code (`_Germany_Launch_Assets` → “Germany Launch Assets”).
3. **Climb the ancestry** to the studio folder. The studio's keyword names the client; the deepest non-year folder between the studio and the job is the **film** (year folders like `2026` are skipped).
4. **Region marker** off the studio's full title: “UNIVERSAL UK” → the UK arm, plain “UNIVERSAL” → International, XYi → none. This shapes the client (“Universal Pictures UK”) and the description prefix (“UK - …”), not the country territory.
5. Assemble the canonical line: `"Film : CODE, Description"`. A folder already in canonical shape is taken verbatim.

The scan's output is compared against the book on the code. New codes are offered for addition. A book row that disagrees with Wrike is flagged with the folder path shown, so you can judge which side is right: **Fix** rewrites the book row to match Wrike; **Keep** records the code in `job_sync_kept` so future scans stop asking about it. The scan is deliberately conservative about what counts as agreement — a book description that contains the Wrike description, or vice versa, is left alone, because the two sides legitimately carry different amounts of the same information.

---

6 Resolving a task — job, film, country

When a task is pulled from Wrike, it arrives with very little: a title, a folder path, and a few custom fields. Everything else is derived in a fixed order (`src/utils/wrikeHelpers.js`, `src/utils/countryCodes.js`):

RESOLUTION ORDER FOR ONE TASK

1. **Job number** — the dedicated "Job Number" custom field first (its suffix preserved, so `XY025716_LUG_D6` survives), else the first `XY\d{5,6}` in the title or path.
2. **Job Book match** — if the code is in the book, the book's canonical string is used whole. The book wins; an unknown code stays bare so the scan can fill it in later.
3. **Country (territory)** — read deliberately, in order: the last token of the task name, then the parent task's name, then the folder the task sits in, then the Country custom field (matched by field id). A country is never guessed from prose — the old rule read "No changes needed" as Norway, and that's exactly what was removed. Multiple countries join into e.g. "UK, Ireland".
4. **Film** — the job number's own `"Film : …"` prefix, else the book's `film_title`, else the task's project name or path, else the title's first token, else a curated mapping. All-caps names are title-cased.
5. **Client** — from the studio in the path plus the resolved country ("Universal Pictures UK" if UK, "Universal Pictures Australia" if Australia, else "Universal Pictures International").
6. **Job Book override** — a known book row replaces *any* guess above for film and client, and upgrades a bare code to the book's canonical string.
7. **Self-registration** — the first time a code is seen it's inserted into the book verbatim; later occurrences only fill blank fields and never overwrite a set one.

The country alias table is ordered weakest-first so real codes can't be shadowed: territory names, then `REGION_ALIASES`, then `MAGI_MARKET_CODES` (what production actually writes), then agreed exceptions. Keys are punctuation-stripped, so MAGI's `BE-FL` arrives as `BEFL` no matter how it was typed.

7 The Wrike sync layer

The boards and the scan need to see Wrike's task tree without calling the Wrike API on every screen. A shared mirror handles that (`src/hooks/useWrikeCache.js`, `src/lib/wrikeEnrich.js`):

1. **Worker owns the token.** All Wrike reads go through the worker, which refreshes and holds the OAuth token.
2. **Two mirrors.** Supabase keeps a team-shared copy (`wrike_tasks_cache`); each browser also keeps a local IndexedDB copy, so the UI can render instantly before the network round-trip.
3. **Delta pulls.** Sync asks Wrike for tasks updated since the last run (with a generous overlap window so nothing edited during the previous run is skipped), enriches them, and upserts. A 15-minute poll gate keeps the call rate low.
4. **Enrichment.** Each task is given a film name and studio from the folder-tree climb, a parsed table if it's a MATRIX task, and pruned custom fields. This is the same derivation the resolution pipeline uses, kept in one place.
5. **Live updates.** Wrike fires webhooks into a Supabase events table; the app subscribes via Realtime, debounces the task ids, and runs the same fetch-and-enrich pass. The poll is the fallback for anything a webhook misses.
6. **Relevance filter.** Only tasks relevant to the motion/launch teams (by keyword, folder, or assignee) are kept in the shared mirror, so the table stays a work set rather than a full dump of the Wrike account.

8 Identity and access

Wrike is the identity provider. The app signs into Supabase with an anonymous session and stamps the Wrike user id into the session's metadata; the database's row-level security then reads that id to scope a user's data to their own rows. Profiles are readable by everyone and editable only by the management allowlist, and the Management pages themselves are gated by the same allowlist. Reads that need to span users (the scan) are routed through the worker rather than loosened on the client.

9 Where the data lives

Table	Purpose
`jobs`	The Job Book — one row per XY code, canonical name, film, client, dates, notes.
`tasks`	Timesheet rows — job, film, country, time, date, source, timelog ids.

`films`	Known films, grouped by studio for the pickers.
`profiles`, `positions`	People, their positions and rates.
`job_categories`, `job_departments`, `project_descriptions`	Lookups used by the entry surfaces and the export.
`job_sync_kept`	XY codes whose scan disagreements were deliberately kept — "stop asking".
`board_now`, `wrike_tasks_cache`, `wrike_sync_meta`, `wrike_webhook_events`	The Wrike mirror and the sync machinery around it.
`wrike_oauth_tokens`, `wrike_webhook_config`	Worker-side Wrike integration state.

Where the code lives

Concern	File
Timesheet store (both surfaces)	`src/hooks/useTasks.js`
Log/edit actions, Wrike pulls	`src/hooks/useTaskActions.js`
Job lookup and registration	`src/hooks/useJobLookup.js`
Wrike scan and film/client derivation	`src/lib/wrikeCampaign.js`
Task → fields resolution	`src/utils/wrikeHelpers.js`
Country resolution	`src/utils/countryCodes.js` + `src/lib/countryField.js`
Wrike sync and enrichment	`src/hooks/useWrikeCache.js`, `src/lib/wrikeEnrich.js`
Wrike API proxy and OAuth	`src/lib/wrikeApi.js`, `worker/index.js`
Database reads	`src/lib/supabaseClient.js`
Management UI	`src/components/Management.jsx`, `src/components/JobBook.jsx`

Engineering notes for the Timesheeter app · How the main sections work and how they stay consistent. For the resolution pipeline in full detail see job-data-technical.pdf; for the plain-English version, job-data-guide.pdf.